A Quadratic Programming-Based Teleoperation Scheme for Haircutting Robots

Author information to be completed

***Abstract*—Teleoperation keeps a human operator in the loop and is a practical way to realize robotic haircutting, whereas heterogeneous master–slave manipulators and dedicated input devices hinder its application. In this article, the teleoperation of robotic haircutting is formulated as a quadratic programming (QP) problem, in which the desired pose supplied by the operator is tracked by the slave manipulator subject to joint-angle and joint-velocity limits, without requiring joint correspondence between the master and slave manipulators. Then, a discrete-time projection neural network is proposed for solving the QP problem online. The discrete-time projection neural network is proven to be globally convergent to the optimal solution at a linear rate, and its output is proven to satisfy the joint limits after any finite number of iterations. Furthermore, the teleoperation with heterogeneous master–slave manipulators is realized on an SO101 master arm and a Piper slave arm with a mannequin head. In addition, a smartphone-based operation mode via a visual Tag is realized, in which the smartphone replaces the dedicated master arm and provides a low-cost solution for the remote control of haircutting robots.**

***Index Terms*—Discrete-time projection neural network, heterogeneous teleoperation, quadratic programming (QP), robotic haircutting, smartphone interface.**

I. Introduction

ROBOTIC haircutting is an emerging personal-care application of robot manipulators, which is motivated by the growing demand for accessible grooming assistance [1], [2]. Different from conventional industrial tasks, haircutting requires a cutting tool to move around the head in the presence of deformable hair, restricted visual access, and sensitive body regions. In recent years, several aspects of robotic haircutting have been investigated, e.g., camera-assisted self-haircutting [3], soft and compliant hair manipulation [4], CNC-inspired task decomposition [5], localization and coverage planning from a mobile robotics perspective [6], and layered safety strategies [7]. Nevertheless, fully autonomous haircutting remains difficult because hair modeling, perception, and personalized styling are still open problems [1], [2]. In view of this, teleoperation that keeps a human operator in the loop is a practical way to bring robotic haircutting into real applications [8].

As a typical human–machine interaction system, telerobotic systems have been widely explored [9]. A variety of telerobotic control schemes have been developed, e.g., neural-learning-based control with collision avoidance [10], shared control for learning and generalizing skills [11], fixed-time composite neural learning control of flexible telerobotic systems [12], finite-time composite learning control under networked time-varying delays [13], and passivity-based stabilization for physical human–(tele)robot interaction [14]. In addition, network transmission protocols [15], live remote interaction platforms [16], and cloud-based high-availability architectures [17] have been developed to support remote operation. Note that most of the above schemes focus on the dynamics, stability, or communication of master–slave systems. If the master and slave manipulators are heterogeneous, i.e., with different joint arrangements and workspaces, then the operator's command cannot be mapped joint by joint, and it has to be realized by the slave subject to its own kinematics and joint limits. Moreover, a dedicated master arm increases the equipment cost of remote operation, which limits the accessibility of telerobotic services.

Quadratic programming (QP) provides a unified way to handle task objectives and physical constraints of robot manipulators. For example, manipulability optimization [18], repetitive motion planning [19], and position-level repetitive motion planning with obstacle avoidance [20] have been formulated as constrained QP problems. Efficient numerical solvers, e.g., ProxQP [21] and ElastiQP [22], have been developed for real-time robot control. Meanwhile, recurrent neural networks (RNNs) have been widely used to solve QP problems owing to their parallel processing capability. A variety of neural solvers have been developed, e.g., penalty-based varying-parameter RNNs [23], adaptive variable-parameter dynamic learning networks [24], gradient-based differential neural networks [25], gradient-based integral neural dynamics [26], reciprocal of exponential varying-parameter RNNs [27], penalty-enhanced error-redefinition RNNs [28], and noise-suppression zeroing neural networks with joint angle constraints [29]. Furthermore, discrete-time neural solvers are compatible with digital controllers, and they have been developed via direct discretization [30], left-and-right two-step discretization [31], and five-instant discretization for the real-time control of a Stewart platform [32]. However, these QP-based schemes are mainly applied to the tracking of predefined trajectories, rather than to human-in-the-loop teleoperation.

In view of the above discussions, QP-based teleoperation is of great importance for robotic haircutting, because the operator's intent and the joint limits of the slave manipulator can be handled within a single optimization problem near the head. Although TeleHairing has recently provided a teleoperation baseline for robotic haircutting with smartphone AprilTag input [8], to the best of our knowledge, a QP-based teleoperation scheme for robotic haircutting with heterogeneous manipulators has not been reported. In this article, a QP-based teleoperation scheme is proposed for robotic

haircutting. Specifically, the desired task-space pose supplied by the operator is converted into slave joint commands by a QP subject to joint-position and joint-velocity bounds, and the QP is solved online by a discrete-time projection neural network. The contributions of this work are summarized as follows.

1) To the best of our knowledge, this is the first work that teleoperates the motion of a manipulator by solving a QP problem online in the complex scenario of robotic haircutting, where the operator's intent and the joint limits of the slave manipulator are handled within a single QP problem.
2) We realize the teleoperation of robotic haircutting with heterogeneous master–slave manipulators. The QP-based scheme converts the operator's Cartesian references into feasible slave joint commands without requiring joint correspondence, and its efficacy is demonstrated by an SO101–Piper experiment on a mannequin head.
3) We realize a simple smartphone-based operation mode via a visual Tag. Via the same QP, the smartphone replaces the dedicated master arm, which provides a low-cost solution for the remote control of haircutting robots.

The remainder of this article is organized as follows. The preliminaries are shown in Section II. In Section III, the optimization problem of haircutting teleoperation is described, and a discrete-time projection neural network is designed. The convergence of the discrete-time projection neural network is proven in Section IV. Experiment results are elaborated in Section V. Conclusions are drawn in Section VI.

II. Preliminaries

A. Notations

In this article, lowercase letters (e.g., a, h) denote scalars, lowercase boldfaced letters (e.g., $\mathbf{u}$, $\mathbf{q}$) denote vectors, and uppercase boldfaced letters (e.g., $\mathbf{H}$, $\mathbf{J}$) denote matrices. $\mathbb{R}^n$ and $\mathbb{R}^n_+$ denote the n-dimensional real space and its nonnegative orthant, respectively. Superscript T denotes the transpose operator; $\mathbf{I}_n$ and $\mathbf{0}_n$ denote the $n \times n$ identity matrix and the n-dimensional zero vector, respectively; $\mathrm{col}[\mathbf{x}_1, \mathbf{x}_2] := (\mathbf{x}_1^T, \mathbf{x}_2^T)^T$; $(\cdot)^+ = \max\{\cdot, 0\}$. Inequalities, $\max\{\cdot\}$, $\min\{\cdot\}$, and $(\cdot)^+$ for vectors are taken elementwise.

B. Projection Neural Network

In [33], a projection neural network is developed as follows and proven to be stable at the optimal solution of optimization problem $\min_{\mathbf{u}\in\Omega} F(\mathbf{u})$:

$$\kappa \frac{d\mathbf{u}}{dt} = -\mathbf{u} + \Pi_\Omega\left(\mathbf{u} - \gamma \nabla F(\mathbf{u})\right) \tag{1}$$

where $\kappa > 0$ is a time constant; $\gamma > 0$ is a scaling parameter; $F : \mathbb{R}^n \to \mathbb{R}$ is a convex objective function; $\Omega = \{\mathbf{u} | \boldsymbol{\ell} \le \mathbf{u} \le \boldsymbol{b}\} \subset \mathbb{R}^n$ is a bound constraint with given lower and upper bounds $\boldsymbol{\ell}$ and $\boldsymbol{b}$; and $\Pi_\Omega(\mathbf{x}) = \arg\min_{\mathbf{y}\in\Omega} \|\mathbf{x} - \mathbf{y}\| = \max\{\min\{\mathbf{x}, \boldsymbol{b}\}, \boldsymbol{\ell}\}$ is the projection function.

III. Main Results

A. Problem Description

Consider a slave manipulator with n_s joints. At the kth time instant, the tracking error is defined as follows:

$$\mathbf{e}_k = \mathbf{r}_{d,k} - \mathbf{r}(\mathbf{q}_{s,k}) \tag{2}$$

where $\mathbf{q}_{s,k} \in \mathbb{R}^{n_s}$ denotes the joint-angle vector of the slave; $\mathbf{r} : \mathbb{R}^{n_s} \to \mathbb{R}^6$ denotes the forward kinematics of the slave, i.e., three positions and three local orientation coordinates; $\mathbf{r}_{d,k} \in \mathbb{R}^6$ denotes the desired pose, which is obtained from either the end-effector pose of the master arm or the pose of the Tag on the smartphone.

To track $\mathbf{r}_{d,k}$ subject to the joint limits of the slave, the joint velocity $\mathbf{u}_k \in \mathbb{R}^{n_s}$ is obtained by solving the following optimization problem:

$$\begin{aligned} \min_{\mathbf{u}_k} \quad & \tfrac{1}{2}\left\|\mathbf{W}(\mathbf{J}_k\mathbf{u}_k - k_e\mathbf{e}_k)\right\|^2 + \tfrac{\lambda}{2}\left\|\mathbf{u}_k\right\|^2 \\ \text{s.t.} \quad & \boldsymbol{\ell}_k \le \mathbf{u}_k \le \boldsymbol{b}_k \end{aligned} \tag{3}$$

where $\mathbf{J}_k = \partial \mathbf{r}(\mathbf{q}_{s,k})/\partial \mathbf{q}_{s,k} \in \mathbb{R}^{6\times n_s}$ is the Jacobian matrix; $\mathbf{W} = \mathrm{diag}(w_p, w_p, w_p, w_r, w_r, w_r)$ is the weighting matrix with $w_p > 0$ and $w_r \ge 0$; $k_e > 0$ is the feedback gain; $\lambda > 0$ is the regularization coefficient; $\boldsymbol{\ell}_k$ and $\boldsymbol{b}_k$ are the lower and upper bounds of $\mathbf{u}_k$, respectively, which are defined as follows:

$$\boldsymbol{\ell}_k = \max\{-\mathbf{v}_{\max}, (\mathbf{q}_{\min} - \mathbf{q}_{s,k})/h_k\} \tag{4}$$

$$\boldsymbol{b}_k = \min\{\mathbf{v}_{\max}, (\mathbf{q}_{\max} - \mathbf{q}_{s,k})/h_k\} \tag{5}$$

where $\mathbf{q}_{\min}$ and $\mathbf{q}_{\max}$ are the lower and upper joint-angle limits, respectively; $\mathbf{v}_{\max}$ is the joint-velocity limit; and $h_k > 0$ is the sampling interval, with $\mathbf{v}_{\max} > \mathbf{0}_{n_s}$. By omitting the constant term, Problem (3) is rewritten as the following QP problem:

$$\begin{aligned} \min_{\mathbf{u}_k} \quad & F_k(\mathbf{u}_k) = \tfrac{1}{2}\mathbf{u}_k^T\mathbf{H}_k\mathbf{u}_k + \mathbf{f}_k^T\mathbf{u}_k \\ \text{s.t.} \quad & \mathbf{g}_k(\mathbf{u}_k) \le \mathbf{0}_{2n_s} \end{aligned} \tag{6}$$

where $\mathbf{H}_k = \mathbf{J}_k^T\mathbf{W}^2\mathbf{J}_k + \lambda\mathbf{I}_{n_s}$, $\mathbf{f}_k = -k_e\mathbf{J}_k^T\mathbf{W}^2\mathbf{e}_k$, and $\mathbf{g}_k(\mathbf{u}_k) = \mathrm{col}[\boldsymbol{\ell}_k - \mathbf{u}_k, \mathbf{u}_k - \boldsymbol{b}_k]$. Let $\Omega_k = \{\mathbf{u} | \mathbf{g}_k(\mathbf{u}) \le \mathbf{0}_{2n_s}\}$ denote the feasible set of Problem (6).

A Lagrangian function of Problem (6) is defined as follows:

$$\mathcal{L}_k(\mathbf{u}, \boldsymbol{\eta}) = F_k(\mathbf{u}) + \boldsymbol{\eta}^T\mathbf{g}_k(\mathbf{u}) \tag{7}$$

where $\boldsymbol{\eta} = \mathrm{col}[\boldsymbol{\eta}^-, \boldsymbol{\eta}^+] \in \mathbb{R}^{2n_s}_+$ is the Lagrange multiplier, and $\boldsymbol{\eta}^-$ and $\boldsymbol{\eta}^+$ correspond to the lower and upper bounds, respectively. The Karush–Kuhn–Tucker (KKT) conditions of Problem (6) are described as follows:

$$\begin{aligned} & \nabla F_k(\mathbf{u}) + \nabla \mathbf{g}_k(\mathbf{u})^T\boldsymbol{\eta} = \nabla F_k(\mathbf{u}) - \boldsymbol{\eta}^- + \boldsymbol{\eta}^+ = \mathbf{0}_{n_s} \\ & \boldsymbol{\eta} \ge \mathbf{0}_{2n_s}, \ \ \mathbf{g}_k(\mathbf{u}) \le \mathbf{0}_{2n_s}, \ \ \mathbf{g}_k(\mathbf{u})^T\boldsymbol{\eta} = 0 \end{aligned} \tag{8}$$

where $\nabla F_k(\mathbf{u}) = \mathbf{H}_k\mathbf{u} + \mathbf{f}_k$. A point $\mathrm{col}[\mathbf{u}, \boldsymbol{\eta}]$ satisfying KKT conditions (8) is said to be a KKT point of Problem (6).

B. Design of a Discrete-Time Projection Neural Network

Since $F_k(\mathbf{u})$ is convex and $\mathbf{g}_k(\mathbf{u})$ is affine, Problem (6) is a convex optimization problem, and $\mathbf{u}$ of a KKT point is an optimal solution to Problem (6) [34]. Next, a discrete-time projection neural network is designed for solving Problem (6) by three steps.

Step 1: KKT conditions (8) are transformed into an equation. Consider the ith component of $\mathbf{u}$. If $\ell_{k,i} < u_i < b_{k,i}$, then $\mathbf{g}_k(\mathbf{u})^T\boldsymbol{\eta} = 0$ yields $\eta_i^- = \eta_i^+ = 0$, and thus $\nabla_i F_k(\mathbf{u}) = 0$. If $u_i = \ell_{k,i} < b_{k,i}$, then $\eta_i^+ = 0$, and thus $\nabla_i F_k(\mathbf{u}) = \eta_i^- \geq 0$. If $u_i = b_{k,i} > \ell_{k,i}$, then $\eta_i^- = 0$, and thus $\nabla_i F_k(\mathbf{u}) = -\eta_i^+ \leq 0$. If $\ell_{k,i} = b_{k,i}$, then $u_i = \ell_{k,i} = b_{k,i}$. As such, $\mathbf{u}$ of a KKT point satisfies the following equation for any $\gamma > 0$:

$$\mathbf{u} = \Pi_{\Omega_k}\left(\mathbf{u} - \gamma\nabla F_k(\mathbf{u})\right). \tag{9}$$

Step 2: In analogy with projection neural network (1) with $\Omega = \Omega_k$ and $F = F_k$, the following continuous-time model is obtained:

$$\kappa\frac{d\mathbf{u}}{dt} = -\mathbf{u} + \mathcal{T}_k(\mathbf{u}) \tag{10}$$

where

$$\mathcal{T}_k(\mathbf{u}) = \Pi_{\Omega_k}\left(\mathbf{u} - \gamma\nabla F_k(\mathbf{u})\right). \tag{11}$$

From (9), the solution of $-\mathbf{u}+\mathcal{T}_k(\mathbf{u}) = \mathbf{0}_{n_s}$ is consistent with $\mathbf{u}$ of a KKT point.

Step 3: Since the slave manipulator is controlled by a digital controller, model (10) is discretized by the Euler forward method with time step $\tau > 0$. Let $\alpha = \tau/\kappa$.

Altogether, a discrete-time projection neural network for solving Problem (6) is proposed as follows:

$$\mathbf{u}_k^{j+1} = \mathbf{u}_k^j + \alpha\left[-\mathbf{u}_k^j + \mathcal{T}_k(\mathbf{u}_k^j)\right] \tag{12}$$

where $\mathbf{u}_k^j \in \mathbb{R}^{n_s}$ is the output neuronal vector at the jth iteration of the kth time instant, and $\alpha \in (0, 1]$ and $\gamma > 0$ are step sizes. In discrete-time projection neural network (12), the gradient term $\gamma\nabla F_k(\mathbf{u}_k^j)$ is designed for reducing the tracking error, and the projection function $\Pi_{\Omega_k}(\cdot)$ is designed for handling the joint-angle and joint-velocity limits.

Discrete-time projection neural network (12) stops if $r_k(\mathbf{u}_k^j) \leq \varepsilon$ or $j = N_{\max}$, where $\varepsilon > 0$ is the steady-state threshold, $N_{\max}$ is the maximum number of iterations, and $r_k(\mathbf{u}) = \|\mathbf{u} - \mathcal{T}_k(\mathbf{u})\|_\infty$ is the residual. Let $\hat{\mathbf{u}}_k$ denote the output of discrete-time projection neural network (12). The joint-angle command sent to the slave is shown as follows:

$$\mathbf{q}_{\mathrm{cmd},k} = \mathbf{q}_{s,k} + h_k\hat{\mathbf{u}}_k. \tag{13}$$

Algorithm 1 details the pseudocode of the proposed discrete-time projection neural network at one time instant.

Algorithm 1 Discrete-Time Projection Neural Network at the kth Time Instant

Require: $\mathbf{q}_{s,k}$, $\mathbf{r}_{d,k}$, $\hat{\mathbf{u}}_{k-1}$ ($\hat{\mathbf{u}}_{-1} = \mathbf{0}_{n_s}$); parameters $\mathbf{W}$, k_e, λ, α, γ, ε, $N_{\max}$.

Ensure: $\mathbf{q}_{\mathrm{cmd},k}$.

Compute $\mathbf{e}_k$, $\mathbf{J}_k$, $\mathbf{H}_k$, $\mathbf{f}_k$, $\boldsymbol{\ell}_k$, and $\boldsymbol{b}_k$;
Set $\mathbf{u}_k^0 = \Pi_{\Omega_k}(\hat{\mathbf{u}}_{k-1})$ and $j = 0$;
while $r_k(\mathbf{u}_k^j) > \varepsilon$ and $j < N_{\max}$ **do**
    $\mathbf{u}_k^{j+1} = \mathbf{u}_k^j + \alpha[-\mathbf{u}_k^j + \mathcal{T}_k(\mathbf{u}_k^j)]$;
    Set $j = j + 1$;
end while
Set $\hat{\mathbf{u}}_k = \mathbf{u}_k^j$ and $\mathbf{q}_{\mathrm{cmd},k} = \mathbf{q}_{s,k} + h_k\hat{\mathbf{u}}_k$.

IV. Convergence Analysis

In this section, the optimality and convergence of discrete-time projection neural network (12) are proven. Throughout this section, $\mathbf{q}_{s,k}$, $\mathbf{r}_{d,k}$, $\mathbf{H}_k$, and $\mathbf{f}_k$ are fixed at the kth time instant. A necessary assumption is provided as follows.

Assumption 1: $\mathbf{q}_{\min} \leq \mathbf{q}_{s,k} \leq \mathbf{q}_{\max}$.

Under Assumption 1, (4) and (5) yield $\boldsymbol{\ell}_k \leq \mathbf{0}_{n_s} \leq \boldsymbol{b}_k$, which implies that Ω_k is nonempty.

Definition 1: $\bar{\mathbf{u}}_k$ is said to be an equilibrium of discrete-time projection neural network (12) if $\bar{\mathbf{u}}_k = \mathcal{T}_k(\bar{\mathbf{u}}_k)$.

Let σ_k and $\bar{\sigma}_k$ denote the minimum and maximum eigenvalues of $\mathbf{H}_k$, respectively.

Theorem 1: Under Assumption 1, Problem (6) has a unique optimal solution $\mathbf{u}_k^\star$, and $\bar{\mathbf{u}}_k = \mathbf{u}_k^\star$.

Proof: According to the definition of equilibria, $\bar{\mathbf{u}}_k$ satisfies the following equation:

$$\bar{\mathbf{u}}_k = \Pi_{\Omega_k}\left(\bar{\mathbf{u}}_k - \gamma\nabla F_k(\bar{\mathbf{u}}_k)\right). \tag{14}$$

Let $\bar{\boldsymbol{\eta}} = \mathrm{col}[\bar{\boldsymbol{\eta}}^-, \bar{\boldsymbol{\eta}}^+]$ with $\bar{\boldsymbol{\eta}}^- = (\nabla F_k(\bar{\mathbf{u}}_k))^+$ and $\bar{\boldsymbol{\eta}}^+ = (-\nabla F_k(\bar{\mathbf{u}}_k))^+$. Then, $\bar{\boldsymbol{\eta}} \geq \mathbf{0}_{2n_s}$ and $\nabla F_k(\bar{\mathbf{u}}_k) - \bar{\boldsymbol{\eta}}^- + \bar{\boldsymbol{\eta}}^+ = \mathbf{0}_{n_s}$. Equation (14) yields that i) $\bar{\mathbf{u}}_k \in \Omega_k$, i.e., $\mathbf{g}_k(\bar{\mathbf{u}}_k) \leq \mathbf{0}_{2n_s}$; ii) if $\bar{u}_{k,i} > \ell_{k,i}$, then $\nabla_i F_k(\bar{\mathbf{u}}_k) \leq 0$, i.e., $\bar{\eta}_i^- = 0$; and iii) if $\bar{u}_{k,i} < b_{k,i}$, then $\nabla_i F_k(\bar{\mathbf{u}}_k) \geq 0$, i.e., $\bar{\eta}_i^+ = 0$. ii) and iii) lead to $\mathbf{g}_k(\bar{\mathbf{u}}_k)^T\bar{\boldsymbol{\eta}} = 0$. In sum, $\mathrm{col}[\bar{\mathbf{u}}_k, \bar{\boldsymbol{\eta}}]$ is a KKT point of Problem (6), and $\bar{\mathbf{u}}_k$ is an optimal solution to Problem (6).

For any nonzero $\mathbf{z} \in \mathbb{R}^{n_s}$, the following inequality holds:

$$\mathbf{z}^T\mathbf{H}_k\mathbf{z} = \|\mathbf{W}\mathbf{J}_k\mathbf{z}\|^2 + \lambda\|\mathbf{z}\|^2 \geq \lambda\|\mathbf{z}\|^2 > 0. \tag{15}$$

It implies that $\sigma_k \geq \lambda > 0$ and $F_k(\mathbf{u})$ is strictly convex. Since Ω_k is nonempty, compact, and convex, Problem (6) has a unique optimal solution $\mathbf{u}_k^\star$. As such, $\bar{\mathbf{u}}_k = \mathbf{u}_k^\star$. Since $\mathbf{g}_k(\mathbf{u})$ is affine, $\mathbf{u}_k^\star$ is $\mathbf{u}$ of a KKT point [34], and it satisfies (9) according to Step 1. Hence, $\mathbf{u}_k^\star$ is the unique equilibrium of discrete-time projection neural network (12), which completes the proof. ∎

Lemma 1 (See [33]): $\|\Pi_{\Omega_k}(\mathbf{y}) - \Pi_{\Omega_k}(\mathbf{x})\| \leq \|\mathbf{y} - \mathbf{x}\|$ $\forall\mathbf{x}, \mathbf{y} \in \mathbb{R}^{n_s}$.

Theorem 2: Under Assumption 1, if $0 < \alpha \leq 1$ and $0 < \gamma < 2/\bar{\sigma}_k$, then discrete-time projection neural network (12) is globally convergent to $\mathbf{u}_k^\star$ with linear convergence rate $\rho_k = 1 - \alpha(1 - c_k) \in [0, 1)$, i.e.,

$$\left\|\mathbf{u}_k^j - \mathbf{u}_k^\star\right\| \leq \rho_k^j\left\|\mathbf{u}_k^0 - \mathbf{u}_k^\star\right\| \tag{16}$$

where $c_k = \max\{|1-\gamma\sigma_k|, |1-\gamma\bar{\sigma}_k|\}$. Moreover, if $\mathbf{u}_k^0 \in \Omega_k$, then $\mathbf{u}_k^j \in \Omega_k$ $\forall j \geq 0$.

Proof: According to Theorem 1, $\bar{\mathbf{u}}_k = \mathbf{u}_k^\star = \mathcal{T}_k(\mathbf{u}_k^\star)$. Then, (12) yields

$$\mathbf{u}_k^{j+1} - \mathbf{u}_k^\star = (1-\alpha)(\mathbf{u}_k^j - \mathbf{u}_k^\star) + \alpha\left[\mathcal{T}_k(\mathbf{u}_k^j) - \mathcal{T}_k(\mathbf{u}_k^\star)\right]. \tag{17}$$

Based on Lemma 1 and (11), one has

$$\left\|\mathcal{T}_k(\mathbf{u}_k^j) - \mathcal{T}_k(\mathbf{u}_k^\star)\right\| \leq \|\mathbf{I}_{n_s} - \gamma\mathbf{H}_k\|_2\left\|\mathbf{u}_k^j - \mathbf{u}_k^\star\right\| \tag{18}$$

where $\|\cdot\|_2$ is the spectral norm of a matrix. Since the eigenvalues of $\mathbf{H}_k$ belong to $[\sigma_k, \bar{\sigma}_k]$, $\|\mathbf{I}_{n_s} - \gamma\mathbf{H}_k\|_2 = c_k$. If $0 < \gamma < 2/\bar{\sigma}_k$, then $0 \le c_k < 1$, and thus $\rho_k \in [0, 1)$ for $0 < \alpha \le 1$. Then, according to (17) and (18), the difference of $V(\mathbf{u}_k^j)$ is derived as follows:

$$\begin{aligned}\Delta V(\mathbf{u}_k^j) &= V(\mathbf{u}_k^{j+1}) - V(\mathbf{u}_k^j) \\ &\le \left[(1-\alpha) + \alpha c_k\right]^2 V(\mathbf{u}_k^j) - V(\mathbf{u}_k^j) \\ &= -(1-\rho_k^2)V(\mathbf{u}_k^j).\end{aligned} \tag{19}$$

Since $1 - \rho_k^2 > 0$, $\Delta V(\mathbf{u}_k^j) \le 0$, and $\Delta V(\mathbf{u}_k^j) = 0$ holds if and only if $\mathbf{u}_k^j = \mathbf{u}_k^\star$. Hence, discrete-time projection neural network (12) is globally asymptotically stable at $\mathbf{u}_k^\star$. Moreover, (19) leads to $V(\mathbf{u}_k^{j+1}) \le \rho_k^2 V(\mathbf{u}_k^j)$, which follows by that $V(\mathbf{u}_k^j) \le \rho_k^{2j} V(\mathbf{u}_k^0)$. Taking the square root yields (16). In summary, the convergence rate is ρ_k.

Now, the invariance of Ω_k is proven. If $\mathbf{u}_k^j \in \Omega_k$, then $\mathcal{T}_k(\mathbf{u}_k^j) \in \Omega_k$. Since Ω_k is convex and $0 < \alpha \le 1$, $\mathbf{u}_k^{j+1} = (1-\alpha)\mathbf{u}_k^j + \alpha\mathcal{T}_k(\mathbf{u}_k^j) \in \Omega_k$. By induction, $\mathbf{u}_k^0 \in \Omega_k$ leads to $\mathbf{u}_k^j \in \Omega_k \ \forall j \ge 0$. ∎

Theorem 3: Under the conditions of Theorem 2, the output $\hat{\mathbf{u}}_k$ of Algorithm 1 and command (13) satisfy the following inequalities, regardless of whether $r_k(\hat{\mathbf{u}}_k) \le \varepsilon$ holds:

$$\|\hat{\mathbf{u}}_k - \mathbf{u}_k^\star\| \le \frac{\sqrt{n_s}}{1 - c_k}\, r_k(\hat{\mathbf{u}}_k) \tag{20}$$

$$-\mathbf{v}_{\max} \le \hat{\mathbf{u}}_k \le \mathbf{v}_{\max} \tag{21}$$

$$\mathbf{q}_{\min} \le \mathbf{q}_{\mathrm{cmd},k} \le \mathbf{q}_{\max}. \tag{22}$$

Proof: Since $\mathbf{u}_k^\star = \mathcal{T}_k(\mathbf{u}_k^\star)$, the triangle inequality and (18) yield

$$\|\hat{\mathbf{u}}_k - \mathbf{u}_k^\star\| \le \|\hat{\mathbf{u}}_k - \mathcal{T}_k(\hat{\mathbf{u}}_k)\| + c_k \|\hat{\mathbf{u}}_k - \mathbf{u}_k^\star\|.$$

Rearranging the above inequality and using $\|\mathbf{x}\| \le \sqrt{n_s}\,\|\mathbf{x}\|_\infty$ yield (20).

Since $\mathbf{u}_k^0 = \Pi_{\Omega_k}(\hat{\mathbf{u}}_{k-1}) \in \Omega_k$, Theorem 2 yields $\hat{\mathbf{u}}_k \in \Omega_k$. In view of (4) and (5), $\hat{\mathbf{u}}_k$ satisfies (21) and

$$(\mathbf{q}_{\min} - \mathbf{q}_{s,k})/h_k \le \hat{\mathbf{u}}_k \le (\mathbf{q}_{\max} - \mathbf{q}_{s,k})/h_k.$$

Multiplying the above inequality by $h_k > 0$ and substituting it into (13) yield (22). ∎

V. Experiment Results

In this section, experiment results are elaborated to evaluate the proposed QP-based teleoperation scheme in terms of tracking accuracy, the effect of joint limits, computational efficiency, and task coverage.

A. Experimental Setup

A six-joint Piper arm is used as the slave, and a tool mounted on the slave is guided around a mannequin head. Two operation modes are tested, i.e., the master-arm mode and the smartphone mode. In both modes, a MuJoCo visualization displays the robot motion and the model-based coverage.

The desired, command-derived, and feedback-derived poses, the commanded and measured joint angles, the computation time, and the number of iterations are recorded at each time instant.

Let $\mathbf{p}_{d,i} \in \mathbb{R}^3$, $\mathbf{p}_{c,i} \in \mathbb{R}^3$, and $\mathbf{p}_{r,i} \in \mathbb{R}^3$ denote the desired, command-derived, and feedback-derived flange positions of the ith sample ($i \in \{1, \ldots, N\}$), respectively, which are expressed in meters in the base frame of the slave. The position errors in millimeters are defined as follows:

$$e_{p,i} = 1000\,\|\mathbf{p}_{r,i} - \mathbf{p}_{d,i}\|, \tag{23}$$

$$e_{p,c,i} = 1000\,\|\mathbf{p}_{c,i} - \mathbf{p}_{d,i}\|. \tag{24}$$

The orientation errors are defined as the magnitudes of the corresponding relative rotation vectors in degrees. Note that the command-derived error results from the feedback correction, regularization, and joint limits, which is different from the error bounded in (20).

TABLE I
Tracking and Solver Statistics in the Master-Arm Mode

Quantity	Mean	RMS	Maximum
Feedback position error (mm)	11.69	14.86	41.78
Command position error (mm)	10.73	14.23	41.24
Feedback orientation error (°)	13.21	17.67	41.77
Command orientation error (°)	12.97	17.60	42.13
Computation time (ms)	4.69	5.30	13.60
Solver iterations	336.65	383.85	500

TABLE II
Mean Tracking Errors Grouped by Commanded Limit Proximity

Mode	Group	Samples	Pos. (mm)	Ori. (°)
Master arm	Away	7938	4.37	1.76
	Near	10079	17.45	22.22
Smartphone	Away	19368	42.73	3.06
	Near	11785	61.06	6.80

B. Heterogeneous Master–Slave Teleoperation

In this experiment, as shown in Fig. 1, an SO101 arm is used as the master, and the Piper arm is used as the slave, which constitute a heterogeneous master–slave system. The operator moves the master arm, and its end-effector pose is used as the desired pose of the slave. The nominal control frequency is 60 Hz, and a total of 18017 samples are recorded over 307.13 s.

Fig. 2 shows the Cartesian reference tracking in the master-arm mode. As shown in Fig. 2(a), the command-derived and feedback-derived positions follow the desired position closely. As shown in Fig. 2(b), the rotation-vector components also follow the desired ones in most segments, whereas evident orientation mismatches appear in some segments. Correspondingly, Fig. 3(a) shows that the orientation error increases in these segments. The statistics of the tracking errors are listed in Table I. Since pose tracking is formulated as a soft objective in Problem (3), a nonzero tracking error does not indicate a failure of the neural network.

To find out the cause of the orientation mismatch, Fig. 4 depicts the commanded and measured joint angles. As shown in Fig. 4(b), the command of joint 5 stays at its upper limit

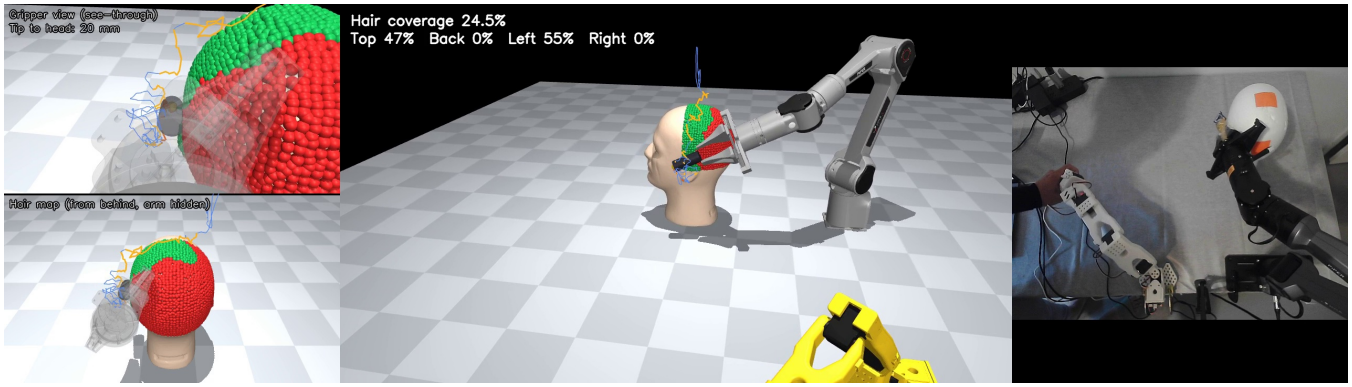


Fig. 1. Frame from the experiment video at 90 s. The frame shows auxiliary simulation views (left), the main robot visualization (center), and the physical camera view (right). The time refers to the video and is not synchronized with the data-log timestamps. Coverage overlays are model-based.

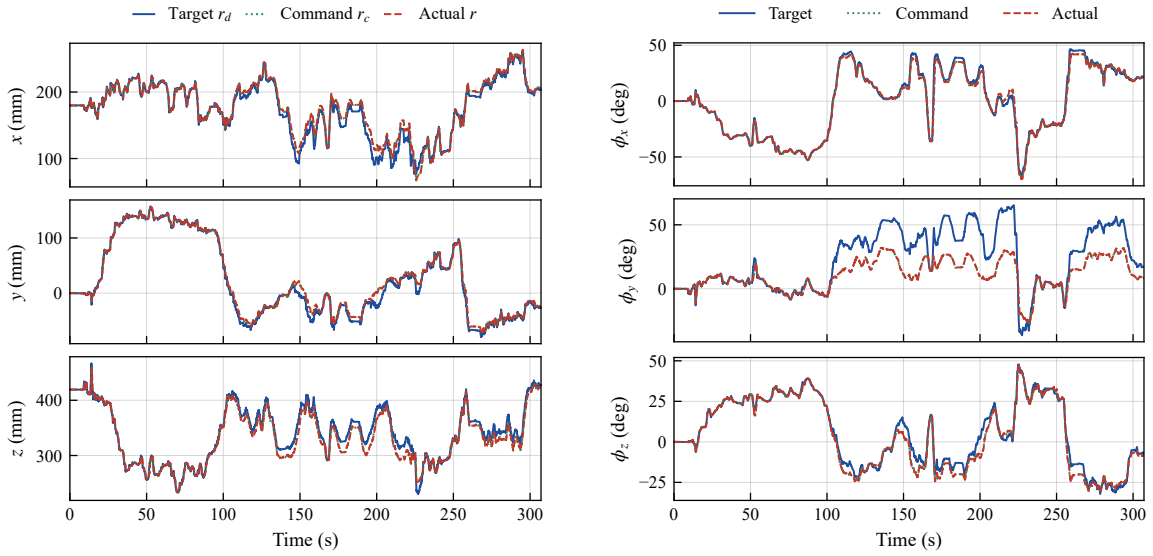


Fig. 2. Cartesian reference tracking in the master-arm mode, where Target, Command, and Actual denote the desired, command-derived, and feedback-derived poses, respectively. (a) Positions. (b) Rotation-vector components relative to the initial desired orientation.

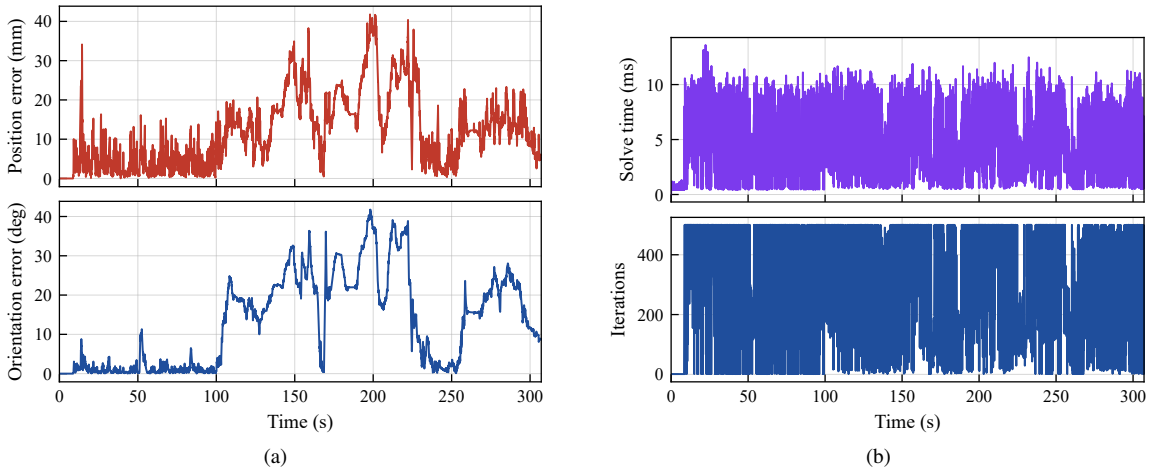


Fig. 3. Tracking errors and computational performance in the master-arm mode. (a) Feedback position and orientation errors. (b) Computation time and number of iterations at each time instant.

for a large portion of the operation. As such, the samples are classified into two groups. If any commanded joint angle is within 0.5° of its limit, i.e., $[-150, 150]$, $[15, 150]$, $[-160, -15]$, $[-100, 100]$, $[-50, 50]$, and $[-100, 100]$ degrees

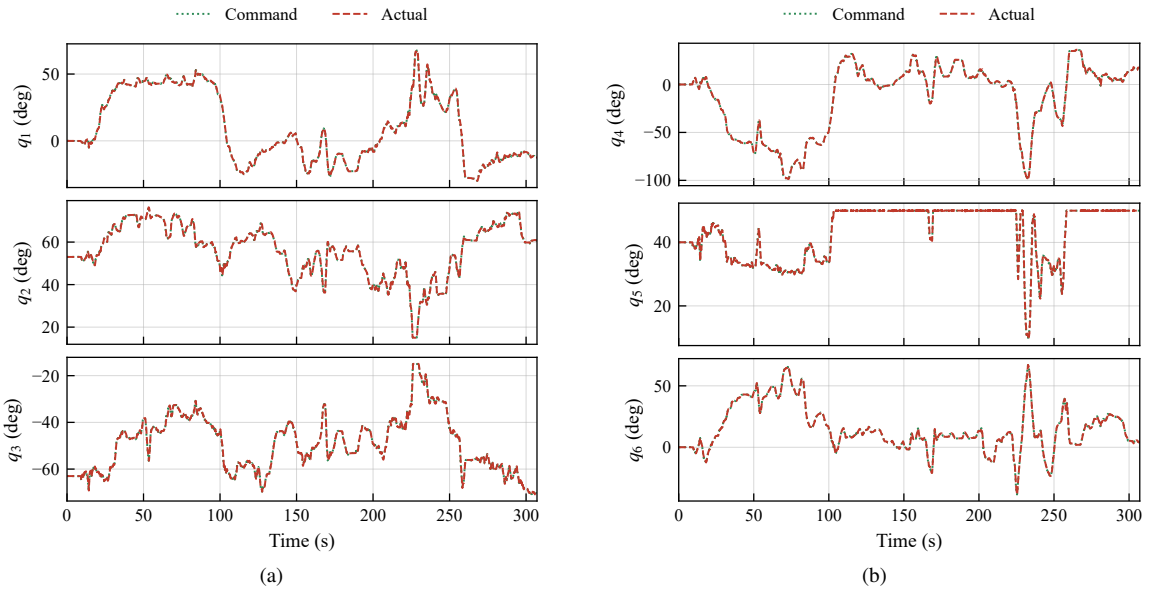


Fig. 4. Commanded and measured joint angles in the master-arm mode. (a) Joints 1–3. (b) Joints 4–6.

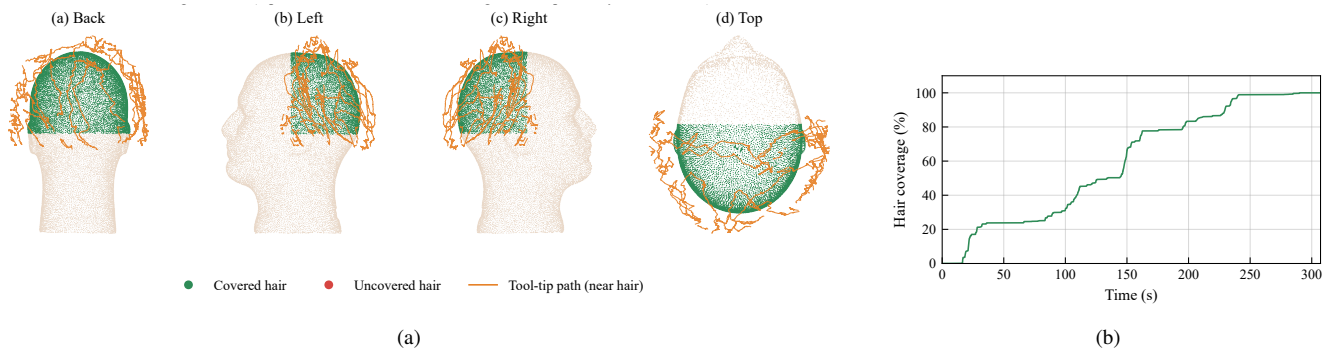


Fig. 5. Geometric coverage of the modeled hair region in the master-arm mode under a 30 mm proximity threshold. (a) Final coverage from four views, where the orange curve denotes the tool-tip path near the hair. (b) Cumulative coverage.

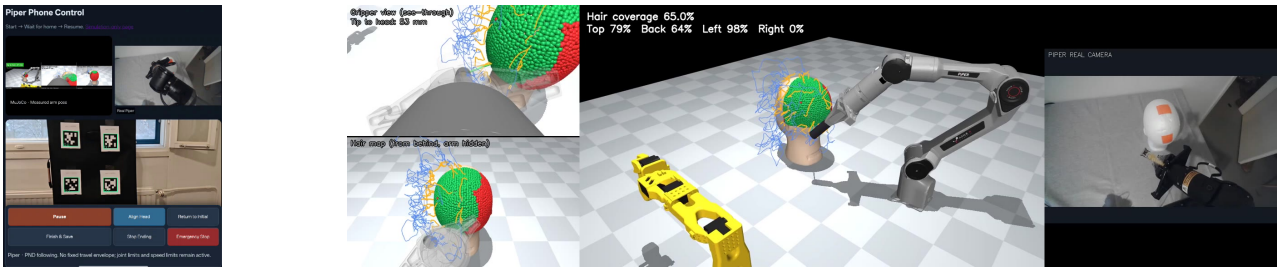


Fig. 6. Smartphone-based teleoperation via Tag. (a) Web interface on the smartphone, which shows the MuJoCo visualization, the camera view of the slave, the camera view of the smartphone with four detected AprilTags, and the control buttons. (b) Frame from the experiment video at 600 s, which shows the auxiliary simulation views (left), the main robot visualization (center), and the physical camera view (right).

for joints 1–6, then the sample is labeled as near-limit. Table II shows that the tracking errors near the limits are much larger than those away from the limits. It indicates that the joint limits restrict the reachable poses of the slave, and the QP sacrifices the tracking accuracy to satisfy the joint limits. Moreover, all commanded joint angles remain within the joint limits, which verifies Theorem 3.

Fig. 3(b) shows that the computation time of the discrete-time projection neural network remains shorter than the sampling interval of 17 ms during the whole operation, where the computation time includes the construction and solution of the QP. The number of iterations reaches $N_{\max} = 500$ in some samples. According to Theorem 3, the joint commands in these samples still satisfy the joint limits.

Fig. 5 shows that the tool tip gradually covers the four regions of the head model, and the geometric coverage finally

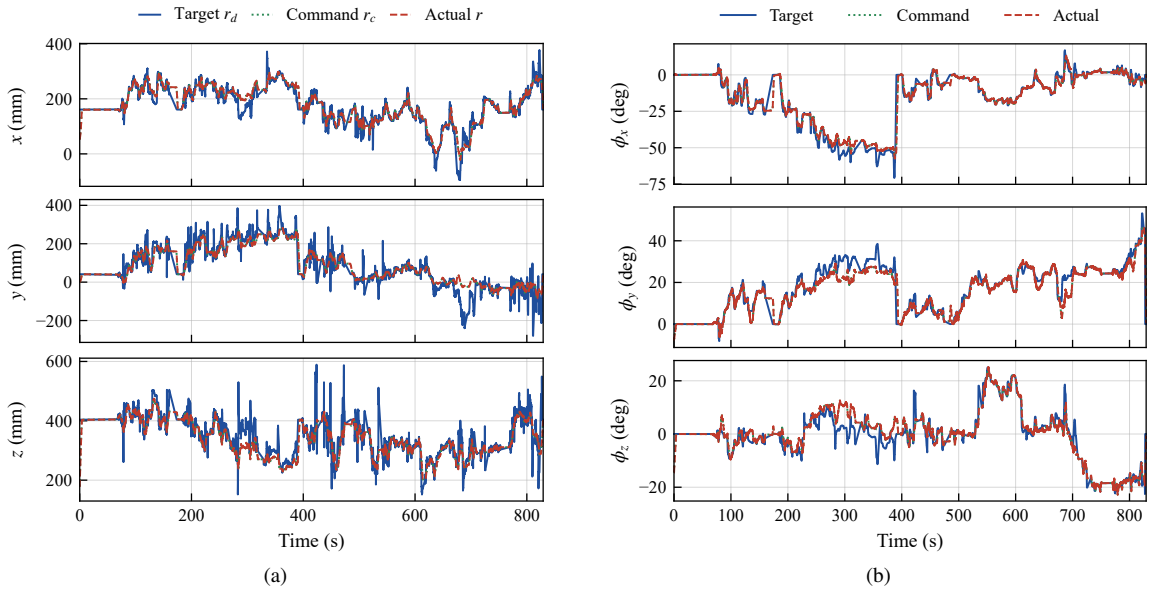


Fig. 7. Cartesian reference tracking in the smartphone mode, where Target, Command, and Actual denote the desired, command-derived, and feedback-derived poses, respectively. (a) Positions. (b) Rotation-vector components relative to the initial desired orientation.

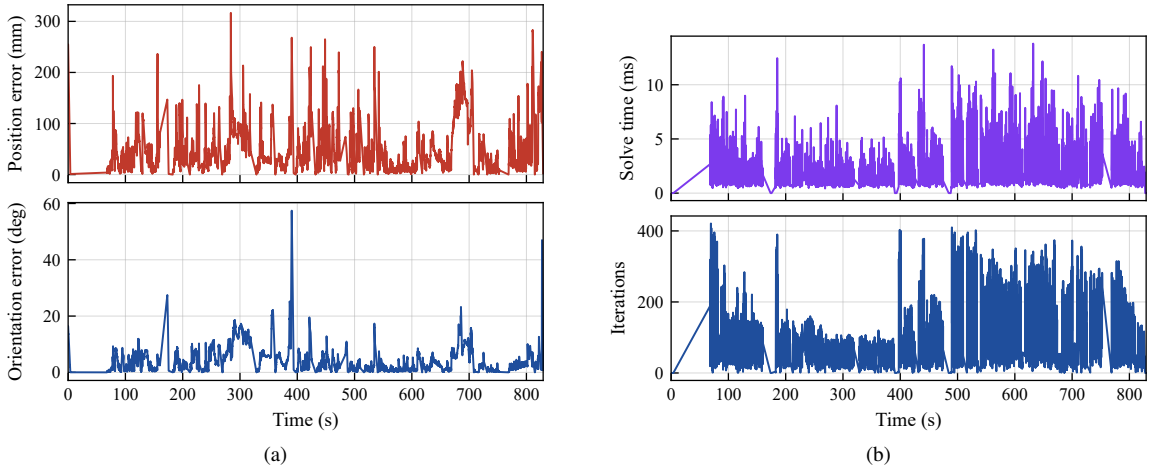


Fig. 8. Tracking errors and computational performance in the smartphone mode. (a) Feedback position and orientation errors. (b) Computation time and number of iterations at each time instant.

reaches 100%, where the coverage is defined as the area of the head model within 30 mm of the tool tip. Note that this metric does not measure the physical hair removal. The above results demonstrate that the proposed scheme realizes the teleoperation of robotic haircutting with heterogeneous master–slave manipulators.

C. Smartphone-Based Teleoperation via Tag

In this experiment, the SO101 master arm is replaced by a smartphone. As shown in Fig. 6(a), the camera of the smartphone observes a board with four AprilTags, and the pose estimated from the Tags is used as the desired pose $\mathbf{r}_{d,k}$ of the Piper slave. The web interface on the smartphone displays the MuJoCo visualization and the camera view of the slave, and it provides the buttons for pausing, homing, and emergency stop. The joint commands are generated by QP (6) and discrete-time projection neural network (12), and no master arm is involved. Fig. 6(b) shows a frame of the experiment video. A total of 31153 samples are recorded over 829.2 s, with a median sampling interval of 24 ms.

TABLE III
COMPARISON BETWEEN THE TWO OPERATION MODES

Quantity	Master arm	Smartphone
Mean position error (mm)	11.69	49.67
Mean orientation error (°)	13.21	4.47
Mean computation time (ms)	4.69	1.88
Maximum computation time (ms)	13.60	13.80
Maximum number of iterations $N_{\max}$	500	1500
Maximum number of iterations used	500	421
Near-limit samples (%)	55.94	37.83
Geometric coverage (%)	100	96.96

Fig. 7 shows the Cartesian reference tracking in the smartphone mode. As shown in Fig. 7(a), the desired position supplied by the hand-held smartphone contains rapid variations,

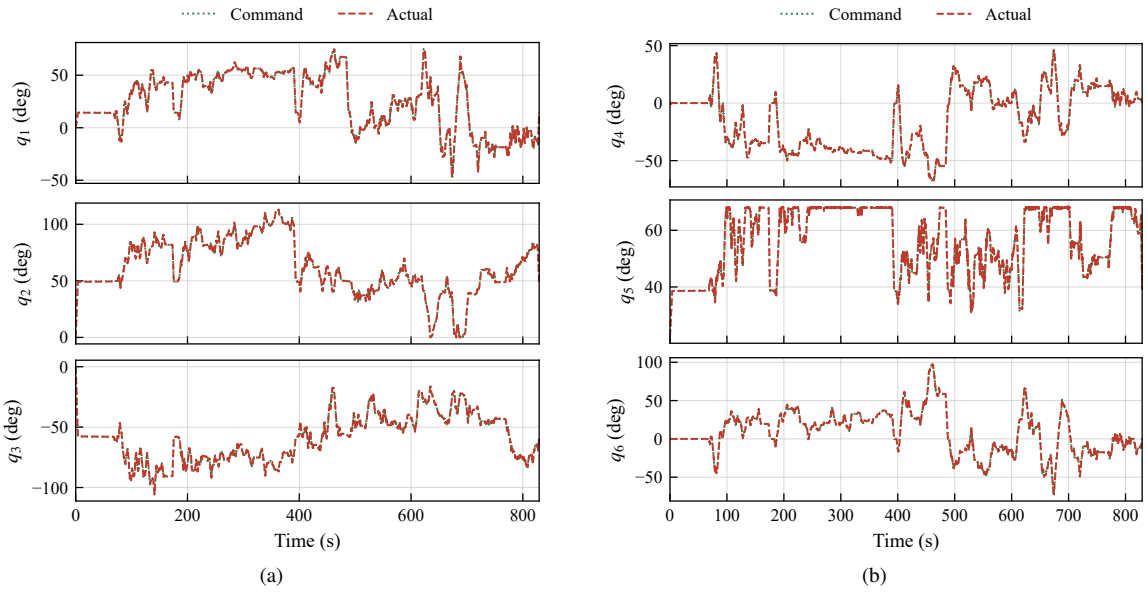


Fig. 9. Commanded and measured joint angles in the smartphone mode. (a) Joints 1–3. (b) Joints 4–6.

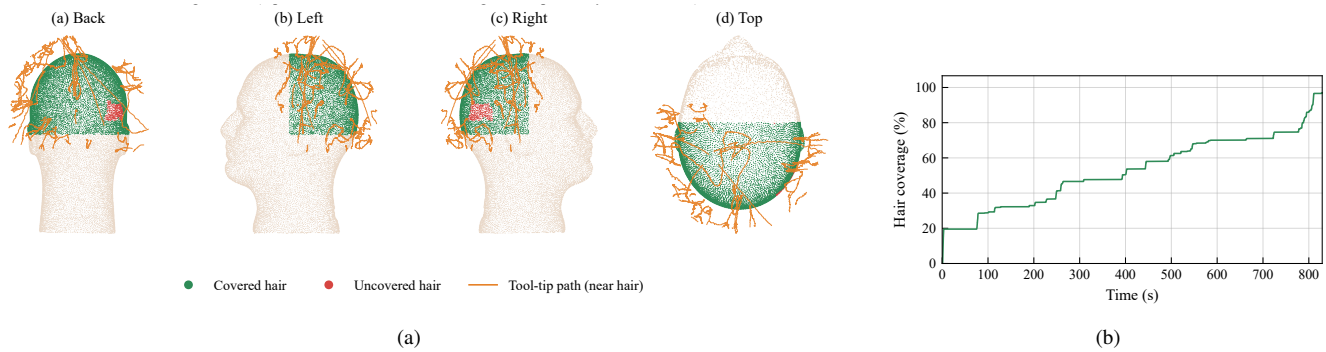


Fig. 10. Geometric coverage of the modeled hair region in the smartphone mode under a 30 mm proximity threshold. (a) Final coverage from four views, where the orange curve denotes the tool-tip path near the hair. (b) Cumulative coverage.

whereas the slave follows a smoother trajectory owing to the joint-velocity limits. As a result, the position error in Fig. 8(a) appears as short peaks. As shown in Fig. 7(b), the rotation-vector components follow the desired ones closely, except for a deviation in the first component between about 150 s and 380 s. Table III shows that, compared with the master-arm mode, the smartphone mode leads to a larger position error but a smaller orientation error. Since the command-derived and feedback-derived positions almost coincide in Fig. 7(a), the position error mainly results from the output of the QP subject to the joint limits, rather than from the low-level control of the slave.

As shown in Fig. 9(b), joint 5 frequently reaches its upper limit, which is similar to the master-arm mode. Table II shows that the tracking errors near the limits are larger than those away from the limits in both modes. The commanded angles of joints 1–4 and 6 remain within their limits. The commanded angle of joint 5 slightly exceeds its upper limit, by at most 0.11°, in some samples. In all these samples, the measured angle of joint 5 has already exceeded the upper limit due to the overshoot of the slave, i.e., Assumption 1 does not hold, and the commanded angle lies between the measured angle and the upper limit, which drives joint 5 back toward its limit.

Fig. 8(b) shows that the computation time remains shorter than the sampling interval during the whole operation, and the number of iterations never reaches $N_{\max} = 1500$. That is, the residual of the discrete-time projection neural network falls below ε at every time instant. Fig. 10 shows that the tool tip gradually covers the head model, and the geometric coverage finally reaches 96.96%, where the uncovered area mainly lies in the back region.

The above results demonstrate that the smartphone replaces the dedicated master arm, and the same QP-based scheme realizes the teleoperation of robotic haircutting with a low-cost device.

VI. Conclusion

In this article, the teleoperation of robotic haircutting has been formulated as a QP problem, in which the desired pose supplied by the operator is tracked by the slave manipulator subject to joint-angle and joint-velocity limits, without requiring joint correspondence between the master and slave manipulators. Then, a discrete-time projection neural network has been proposed for solving the QP problem online. The

discrete-time projection neural network has been proven to be globally convergent to the optimal solution at a linear rate, and its output has been proven to satisfy the joint limits after any finite number of iterations. Furthermore, the teleoperation with heterogeneous master–slave manipulators has been realized on an SO101–Piper system. In addition, a smartphone-based operation mode via a visual Tag has been realized, which provides a low-cost solution for the remote control of haircutting robots. Further investigations may aim at the filtering of the desired pose supplied by the smartphone, the evaluation under remote communication, the reduction of the tracking error caused by joint limits, and the evaluation of physical haircutting outcomes.

REFERENCES

[1] S. Li, *Haircutting Robots*. Cham, Switzerland: Springer, 2025.

[2] A. T. Khan and S. Li, "Robotic haircutting systems: A survey of methods, challenges, and hair modeling insights," *IEEE J. Sel. Areas Sensors*, vol. 3, pp. 104–112, 2026.

[3] K. Futami, T. Terada, and M. Tsukamoto, "A system for supporting self-haircuts using camera equipped robot," in *Proc. Int. Conf. Adv. Mobile Comput. Multimedia*, Dec. 2014.

[4] U. Yoo, N. Dennler, E. Xing, M. Mataric, S. Nikolaidis, J. Ichnowski, and J. Oh, "Soft and compliant contact-rich hair manipulation and care," in *Proc. ACM/IEEE Int. Conf. Human-Robot Interact. (HRI)*, pp. 610–619, Mar. 2025.

[5] A. T. Khan and S. Li, "CNC-inspired robotic hair cutting: A comprehensive survey on precision personal care automation," *J. Artif. Intell. Autom.*, vol. 1, no. 1, Jan. 2026, Art. no. 2.

[6] Z. Huang, A. T. Khan, and S. Li, "Haircutting robots: A mobile robotics perspective," *Int. J. Syst. Sci.*, DOI 10.1080/00207721.2026.2687894, 2026.

[7] Z. Huang, A. Vilkki, S. Li, and J. Röning, "Safety in robotic haircutting," *J. Field Robot.*, DOI 10.1002/rob.70305, 2026.

[8] Z. Huang, A. Vilkki, J. Huang, B. Liao, C. Luo, L. Angrisani, and S. Li, "TeleHairing: A teleoperation baseline for robotic haircutting," 2026, arXiv:2610.07096v1.

[9] G. Li, Q. Li, C. Yang, Y. Su, Z. Yuan, and X. Wu, "The classification and new trends of shared control strategies in telerobotic systems: A survey," *IEEE Trans. Haptics*, vol. 16, no. 2, pp. 118–133, Apr.–Jun. 2023.

[10] C. Yang, X. Wang, L. Cheng, and H. Ma, "Neural-learning-based telerobot control with guaranteed performance," *IEEE Trans. Cybern.*, vol. 47, no. 10, pp. 3148–3159, Oct. 2017.

[11] X. Shao and L. Xie, "A telerobotic shared control architecture for learning and generalizing skills in unstructured environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 15 953–15 965, 2025.

[12] Y. Yang, H. Jiang, L. Gan, C. Hua, and J. Li, "Fixed-time composite neural learning control of flexible telerobotic systems," *IEEE Trans. Cybern.*, vol. 54, no. 6, pp. 3602–3614, Jun. 2024.

[13] Y. Yang, H. Jiang, C. Hua, and J. Li, "Finite-time composite learning control for nonlinear teleoperation systems under networked time-varying delays," *Sci. China Inf. Sci.*, vol. 67, no. 6, Jun. 2024, Art. no. 162203.

[14] P. Paik, S. Thudi, and S. F. Atashzar, "Power-based velocity-domain variable structure passivity signature control for physical human-(tele)robot interaction," *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 386–398, Feb. 2023.

[15] X. Liu, Y. Xue, Y. Wang, Z. Song, J. Ye, X. Ma, and N. Xi, "A new network transmission protocol for Internet-based telerobotic operations," *IEEE Internet Things J.*, vol. 12, no. 24, pp. 54 116–54 132, Dec. 2025.

[16] X. Wang, Y. Wu, J. Yu, Y. Wang, E. Liu, D. Zhao, and G. Liu, "TeleRobot: Design and implementation of a live remote interaction platform for robots," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, pp. 408–412, Oct. 2023.

[17] S. Gu, O. M. Chisom, J. Yang, and D. Yoon, "Cloud-based remote-controlled robot system: A Kubernetes-orchestrated approach to high availability," *Expert Syst. Appl.*, vol. 285, 2025, Art. no. 127860.

[18] L. Jin, S. Li, H. M. La, and X. Luo, "Manipulability optimization of redundant manipulators using dynamic neural networks," *IEEE Trans. Ind. Electron.*, vol. 64, no. 6, pp. 4710–4720, Jun. 2017.

[19] Z. Xie, L. Jin, X. Du, X. Xiao, H. Li, and S. Li, "On generalized RMP scheme for redundant robot manipulators aided with dynamic neural networks and nonconvex bound constraints," *IEEE Trans. Ind. Informat.*, vol. 15, no. 9, pp. 5172–5181, Sep. 2019.

[20] W. Li, Z. Yi, R. Rao, D. Guo, Y. Pan, and K. Huang, "A quadratic programming solution to position-level repetitive motion planning and obstacle avoidance of joint-constrained robots," *IEEE Trans. Ind. Electron.*, vol. 72, no. 2, pp. 1860–1870, Feb. 2025.

[21] A. Bambade, F. Schramm, S. El-Kazdadi, S. Caron, A. Taylor, and J. Carpentier, "ProxQP: An efficient and versatile quadratic programming solver for real-time robotics applications and beyond," *IEEE Trans. Robot.*, DOI 10.1109/TRO.2025.3577107, 2025.

[22] D. Morton and J. Arrizabalaga, "ElastiQP: An always-feasible QP solver for constrained robot control," 2026, arXiv:2609.19080v1.

[23] Z. Zhang, S. Yang, and L. Zheng, "A penalty strategy combined varying-parameter recurrent neural network for solving time-varying multi-type constrained quadratic programming problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 7, pp. 2993–3004, Jul. 2021.

[24] Z. Zhang, X. Sun, X. Li, and Y. Liu, "An adaptive variable-parameter dynamic learning network for solving constrained time-varying QP problem," *Neural Netw.*, vol. 184, Apr. 2025, Art. no. 106968.

[25] B. Liao, Y. Zeng, T. Wang, and Z. Li, "Gradient-based differential neural network to time-varying constrained quadratic programming," *Expert Syst. Appl.*, vol. 264, 2025, Art. no. 125893.

[26] Z. Sun, C. Yu, J. Zhou, L. Jin, and G. Wang, "Gradient-based integral neural dynamics model for solving time-varying quadratic programming: Application to manipulator," *IEEE Trans. Ind. Informat.*, DOI 10.1109/TII.2026.3709622, 2026.

[27] Y. Wang, Y. Han, Z. Tang, W. Liu, and P. Wang, "Reciprocal of exponential varying-parameter RNN solving repetitive tracking control problems with tolerance of random initial error compounded with noise perturbation," *IEEE Trans. Ind. Informat.*, vol. 19, no. 11, pp. 10 998–11 007, Nov. 2023.

[28] T. Zhang and S. Yang, "Penalty-enhanced error-redefinition RNN for time-varying QP problems with multiple constraints and robot arm applications," *Nonlinear Dyn.*, vol. 113, pp. 23 259–23 283, 2025.

[29] Z. Sun, Y. Fei, S. Tang, X. Xiao, J. Luo, and K. Liu, "A noise suppression zeroing neural network for trajectory tracking with joint angle constraints of mobile manipulator," *Eng. Appl. Artif. Intell.*, vol. 133, 2024, Art. no. 108173.

[30] Y. Shi, W. Su, X. Cao, M. Liu, J. Wang, and D. K. Gerontitis, "Novel RNN and its enhanced variant based on direct discretization approach for discrete-form time-dependent quadratic programming," *IEEE Trans. Autom. Sci. Eng.*, vol. 23, pp. 1399–1410, 2026.

[31] M. Yang, Z. Zhu, J. Li, P. Guo, H. Zhang, and S. Li, "Novel left-and-right two-step discrete ZNN model for time-dependent nonlinear optimization with equality and inequality constraints applied to robot manipulator," *IEEE Trans. Ind. Electron.*, DOI 10.1109/TIE.2026.3692861, 2026, early access.

[32] Y. Shi, W. Sheng, J. Wang, L. Jin, B. Li, and X. Sun, "Real-time tracking control and efficiency analyses for Stewart platform based on discrete-time recurrent neural network," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 8, pp. 5099–5111, Aug. 2024.

[33] Y. Xia, H. Leung, and J. Wang, "A projection neural network and its application to constrained optimization problems," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 49, no. 4, pp. 447–458, Apr. 2002.

[34] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.